

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: Deploy a Microservice Application on EKS

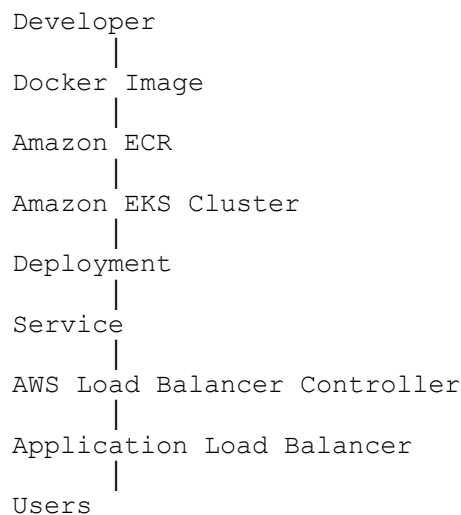
Scenario

Your company has containerized a Java application and wants to deploy it on Kubernetes with high availability, rolling updates, and auto-healing.

How would you deploy it on Amazon EKS?

Solution

Architecture



Steps

1. Create an EKS cluster.
2. Create managed node groups (or use Fargate).
3. Push the Docker image to ECR.
4. Create a Deployment.
5. Create a Service.
6. Configure Ingress using AWS Load Balancer Controller.
7. Access the application through ALB.

Kubernetes Objects Used

- Namespace
 - Deployment
 - ReplicaSet
 - Pods
 - Service
 - Ingress
 - ConfigMap
 - Secret
-

Scenario 2: Pods Are Continuously Restarting (CrashLoopBackOff)

Scenario

After deployment, all Pods enter **CrashLoopBackOff**.

How would you troubleshoot?

Solution

Step 1

Check Pod status.

```
kubectl get pods
```

Step 2

Describe Pod.

```
kubectl describe pod <pod-name>
```

Look for:

- Events
 - Scheduling failures
 - Liveness probe failures
 - Image errors
-

Step 3

Check logs.

```
kubectl logs <pod-name>
```

Possible findings:

- Application exception
 - Missing environment variable
 - Database connection failure
 - Port already in use
-

Step 4

Verify

- Secrets
 - ConfigMaps
 - Image version
 - Resource limits
-

Common Causes

- Wrong image
 - Application crash
 - Missing secret
 - Database unavailable
 - Incorrect startup command
 - OOMKilled
-

Scenario 3: Pods Are Pending

Scenario

Pods never start and remain in **Pending** state.

How do you investigate?

Solution

```
kubectl describe pod
```

Common reasons:

No Worker Nodes

0/3 nodes available

Insufficient CPU

Insufficient CPU

Insufficient Memory

Insufficient Memory

Node Selector Mismatch

```
nodeSelector:  
  type: gpu
```

No node has this label.

Taints

Node

NoSchedule

Pod has no toleration.

PVC Not Bound

Storage unavailable.

Resolution

- Scale node group
- Fix resource requests
- Correct node labels
- Add tolerations
- Resolve storage issues

Scenario 4: Upgrade an EKS Cluster with Zero Downtime

Scenario

Production cluster is running Kubernetes 1.30. AWS recommends upgrading to 1.31.

How would you perform the upgrade?

Solution

Step 1

Check compatibility.

- Add-ons
 - CSI Driver
 - CNI Plugin
 - CoreDNS
 - kube-proxy
-

Step 2

Upgrade Control Plane

AWS Console or CLI

Step 3

Upgrade Managed Node Group

Create new nodes.

Drain old nodes.

```
kubect1 drain node-name
```

Step 4

Upgrade Add-ons

- VPC CNI
 - CoreDNS
 - kube-proxy
 - EBS CSI Driver
-

Step 5

Validate

```
kubectl get nodes  
kubectl get pods
```

Best Practices

- Backup manifests.
 - Test in staging first.
 - Upgrade one minor version at a time.
 - Monitor workloads throughout.
-

Scenario 5: Deploy Application Without Downtime

Scenario

New application version must be deployed while users continue accessing the application.

Solution

Use **Rolling Update**.

Deployment

```
strategy:  
  type: RollingUpdate
```

Example

Current Pods

5

New Version

Start 1

↓

Health Check

↓

Delete 1 Old Pod

↓

Repeat

No downtime.

Rollback

```
kubectl rollout undo deployment app
```

Scenario 6: Auto Scale Pods Based on CPU Usage

Scenario

Traffic increases every morning.

Pods should automatically scale.

Solution

Use

Horizontal Pod Autoscaler.

CPU > 70%

↓

Increase Pods

↓

CPU < 30%

↓

Decrease Pods

Example

```
kubectl autoscale deployment app --cpu-percent=70 --min=2 --max=10
```

Requirements

- Metrics Server installed
 - Resource requests defined
-

Benefits

- Cost optimization
 - Better performance
 - Automatic scaling
-

Scenario 7: Auto Scale Worker Nodes

Scenario

HPA creates more Pods, but there are not enough nodes to schedule them.

Solution

Use **Cluster Autoscaler** (or Karpenter in many modern EKS environments).

Architecture

Pods Pending

↓

Cluster Autoscaler

↓

Increase EC2 Nodes

↓

Pods Scheduled

Flow

1. Pod Pending
 2. Autoscaler detects insufficient capacity
 3. Auto Scaling Group increases desired capacity
 4. New EC2 joins cluster
 5. Pods scheduled
-

Scenario 8: Securely Provide Database Credentials

Scenario

Developers stored passwords inside Deployment YAML.

Security team rejected it.

Solution

Store credentials in:

- Kubernetes Secrets
- AWS Secrets Manager (recommended for production)

Example

Secrets Manager

↓

IAM Role for Service Account (IRSA)

↓

Pod

↓

Application

Benefits

- No hardcoded passwords
 - IAM-controlled access
 - Secret rotation support
-

Scenario 9: External Users Cannot Access the Application

Scenario

Pods are healthy, but users receive **502** or **504** errors.

How would you troubleshoot?

Solution

Check sequentially:

Pods

```
kubectl get pods
```

Service

```
kubectl get svc
```

Endpoints

```
kubectl get endpoints
```

Ingress

```
kubectl get ingress
```

AWS Load Balancer Controller

```
kubectl get pods -n kube-system
```

Target Group

Verify:

- Healthy targets
 - Correct health check path
 - Security Groups
 - Port mappings
-

Common Issues

- Wrong Service selector
 - Incorrect targetPort
 - Failed readiness probe
 - ALB health check mismatch
 - Security Group blocks traffic
-

Scenario 10: Deploy Multiple Microservices

Scenario

Deploy

- User Service
 - Order Service
 - Payment Service
 - Product Service
-

Solution

Architecture

Namespace

```
|— User Deployment
|— Order Deployment
|— Product Deployment
|— Payment Deployment
```

Ingress

/users

↓

User Service

/orders

↓

Order Service

/products

↓

Product Service

Each microservice has

- Deployment
- Service
- HPA
- ConfigMap
- Secret

Scenario 11: A Worker Node Suddenly Becomes NotReady

Scenario

One node changes to **NotReady**, and the pods running on it become unavailable.

How would you troubleshoot?

Solution

Check node status

```
kubectl get nodes
```

Describe node

```
kubectl describe node <node-name>
```

Verify EC2 instance

Check:

- Instance health
- CPU and memory
- Disk usage
- Network connectivity

Check kubelet

```
sudo systemctl status kubelet
```

Check container runtime

```
sudo systemctl status containerd
```

Resolution

- Restart kubelet if required.
 - Replace the failed node.
 - Let EKS reschedule pods to healthy nodes.
 - Investigate the root cause (resource exhaustion, networking, hardware, etc.).
-

Scenario 12: Restrict Communication Between Microservices

Scenario

The Payment service should only accept traffic from the Order service. Other pods must not access it.

Solution

Use **Network Policies**.

Example policy:

```
Allow:
Order Pods
  |
  v
Payment Pods

Deny:
All Other Pods
```

Requirements:

- A CNI plugin that supports Network Policies (such as Amazon VPC CNI with Network Policy support enabled or another compatible CNI).

Benefits

- Zero Trust networking
 - Reduced attack surface
 - Better security compliance
-

Scenario 13: Run Batch Jobs Every Night

Scenario

A report generation job must run every day at midnight.

Solution

Use **CronJob**.

Example schedule:

```
schedule: "0 0 * * *"
```

Flow:

Midnight

↓

CronJob

↓

Job

↓

Pod

↓

Complete

Benefits

- Automated scheduling
- Retry support

- Easy monitoring

Scenario 14: Applications Need Persistent Storage

Scenario

Your application stores uploaded files. After pod restarts, the files disappear.

Solution

Use Persistent Volumes.

Architecture

Pod

↓

Persistent Volume Claim

↓

Persistent Volume

↓

Amazon EBS

For shared storage across multiple pods or AZs, consider Amazon EFS where appropriate.

Benefits

- Data survives pod restarts
- Independent of pod lifecycle
- Easier backup and recovery

Scenario 15: Production Deployment Failed

Scenario

A new deployment causes:

- 503 errors
- Failed health checks
- Pods restarting

How would you handle the incident?

Solution

Step 1

Check rollout status.

```
kubectl rollout status deployment app
```

Step 2

Check logs.

```
kubectl logs
```

Step 3

Describe pods.

```
kubectl describe pod
```

Step 4

Verify:

- ConfigMaps
 - Secrets
 - Image version
 - Readiness probe
 - Liveness probe
 - Resource requests and limits
-

Step 5

Rollback immediately.

```
kubectl rollout undo deployment app
```

Step 6

Perform Root Cause Analysis (RCA)

Document:

- Timeline
- Root cause
- Impact
- Resolution
- Preventive actions

EKS Interview Cheat Sheet

Component	Purpose
EKS Cluster	Managed Kubernetes control plane
Managed Node Group	AWS-managed worker nodes
Fargate Profile	Serverless execution for pods
Deployment	Manages stateless applications
StatefulSet	Manages stateful applications
DaemonSet	Runs one pod per node
ReplicaSet	Maintains desired pod count
Service	Exposes pods internally or externally
Ingress	HTTP/HTTPS routing to services
AWS Load Balancer Controller	Creates ALBs/NLBs for Kubernetes ingress/services
ConfigMap	Stores non-sensitive configuration
Secret	Stores sensitive configuration
IRSA	Grants IAM permissions to pods without static AWS credentials
HPA	Scales pods automatically
Cluster Autoscaler / Karpenter	Scales worker nodes
Persistent Volume (PV)	Physical storage resource
Persistent Volume Claim (PVC)	Storage request by a pod
Amazon EBS CSI Driver	Provides persistent block storage for pods
Amazon EFS CSI Driver	Provides shared file storage for multiple pods
CoreDNS	Cluster DNS service
VPC CNI	Assigns VPC IP addresses to pods
kube-proxy	Handles Kubernetes service networking

